SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science & Engineering

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code	**CSA4315**	**Course Title**	**Internet Programming**
CO Assessed	**CO1**	**Assessment**	**Assessment Tool 1: Application Based Questions**
Weightage	**40% of CIA**	**Total Marks**	**100**
CO1 Statement	*Explain the fundamentals of web architecture, client-server communication, Internet protocols, HTTP request and response messages, and configure LAMP/WAMP environments for web application development and deployment.* **(BL3)**		
Student Name	**K Sumanth Kumar Reddy**	**Reg. No.**	**192425222**
Section/Batch	**Slot D / 2024**	**Date**	**20-10-2026**

Code of Conduct:

I, K Sumanth Kumar Reddy (192425222), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

K Sumanth

APPLICATION-BASED QUESTION

15. A PHP Website Works Correctly on a Developer's Windows Computer but Fails After Deployment to a Linux Server

1. Introduction

PHP (Hypertext Preprocessor) is a widely used server-side scripting language for developing dynamic websites and web applications. It allows developers to create interactive web pages, process forms, manage user accounts, connect to databases, upload files, and generate dynamic content. PHP applications are commonly developed on Windows computers and deployed to Linux servers because Linux is widely used for hosting websites due to its stability, flexibility, performance, and cost-effectiveness.

During the development stage, programmers may use software packages such as XAMPP, WAMP, or Laragon to run PHP applications locally. These packages provide a local web server, PHP interpreter, and database server. After development and testing, the application is transferred to a production server running Linux.

However, a PHP website that works correctly on a Windows computer may fail after deployment to Linux. The application might display a blank page, produce a server error, fail to connect to the database, be unable to upload files, or return a 404 Not Found error. These problems occur because Windows and Linux differ in their file systems, permission models, software versions, server configurations, and system environments.

The main objective of this analysis is to identify the possible causes of deployment failures and explain the appropriate methods for diagnosing and resolving them. Understanding these differences helps developers build reliable, portable, secure, and maintainable web applications.

2. Analysis of Possible Causes

2.1 Operating System Differences

Windows and Linux handle files and directories differently. One important difference is that Linux file systems are usually case-sensitive, whereas Windows file systems are generally case-insensitive.

For example, consider the following PHP statement:

include("Header.php");

If the actual file name is header.php, the code may work on Windows but fail on Linux because the file names do not match exactly.

Linux also uses different path separators and directory structures. Windows paths commonly use backslashes, while Linux paths use forward slashes.

Solution:

- Check file names and directory names for correct capitalization.
- Use consistent file naming throughout the application.
- Use suitable relative paths or PHP path constants.
- Test the application on a Linux environment before deployment.

2.2 File and Directory Permissions

Linux uses a permission system to control who can read, write, and execute files. These permissions are assigned to the file owner, group, and other users.

A PHP website may require permission to read source files, upload images, write log files, or store user-uploaded documents. If the web server does not have the required permissions, the application may fail.

For example, an upload feature may work on Windows but fail on Linux because the upload directory is not writable by the web server.

Solution:

- Check the ownership of files and directories.
- Provide the minimum required read and write permissions.
- Ensure that upload and cache directories are writable when necessary.
- Avoid giving unrestricted permissions such as 777, as they create security risks.

2.3 Software Version Differences

The Windows development computer and Linux server may use different versions of PHP, database software, or web servers. Some functions and features available in one PHP version may be deprecated or unavailable in another version.

For example, an application developed using PHP 8.3 may encounter compatibility problems if the Linux server runs an older PHP version. Missing PHP extensions can also cause errors.

Common examples include missing database drivers, image-processing extensions, or other required PHP modules.

Solution:

- Check the PHP version on both systems.
- Install a compatible PHP version on the Linux server.
- Enable the required PHP extensions.
- Verify compatibility with the database and web server.
- Use dependency management tools such as Composer to manage PHP libraries.

2.4 Configuration Differences

PHP applications depend on configuration settings that control how they execute. The Windows development environment may have different settings from the Linux production server.

Important settings include:

- php.ini: Controls PHP behavior and extensions.
- display_errors: Determines whether errors are displayed to users.
- upload_max_filesize: Sets the maximum size of uploaded files.
- post_max_size: Limits the size of POST request data.
- Database configuration: Defines database host, username, password, and database name.

For example, a website may allow large file uploads on Windows but reject them on Linux because the server has a smaller upload limit.

Solution:

- Compare the configuration files of both environments.
- Set appropriate upload and execution limits.
- Configure database credentials correctly.
- Store environment-specific settings securely.
- Disable detailed error display in production and use server-side logs for debugging.

2.5 Database Connection Problems

A PHP website may work on Windows because the database is running locally, but fail on Linux because the database server, database name, credentials, or network configuration differs.

For example, the Windows application may connect to a local MySQL database, while the Linux server uses a separate database server with different credentials.

Other possible causes include missing database drivers, incorrect hostnames, and insufficient database privileges.

Solution:

- Verify the database server address and port.
- Check the database name, username, and password.
- Ensure the required PHP database extension is installed.
- Confirm that the database user has the necessary permissions.
- Test the connection independently before running the application.

2.6 Web Server Differences

Windows development environments commonly use tools such as XAMPP, while Linux production servers often use Apache or Nginx. These web servers may handle URL rewriting, directory access, and PHP execution differently.

For example, a website using clean URLs may work on Windows but return a 404 error on Linux if URL rewriting is not enabled or the server configuration is incorrect.

Solution:

- Verify that Apache or Nginx is configured correctly.
- Enable the required rewrite rules.
- Check document root settings.
- Confirm that the server can execute PHP files.
- Review web server error logs to identify configuration problems.

3. Troubleshooting Procedure

The developer should follow a systematic procedure to identify and fix the problem.

1. Check error logs: Examine PHP and web server logs for error messages.
2. Verify file names: Check capitalization and file paths.
3. Check permissions: Ensure that the web server can access the required files and directories.
4. Verify software versions: Compare PHP versions and installed extensions.
5. Check configuration: Review PHP settings, environment variables, and server configuration.
6. Test database connectivity: Verify database credentials and privileges.
7. Test the application: Test all website features after making corrections.

4. Example: Troubleshooting an Online Shopping Website

Consider an online shopping application developed on Windows using PHP and MySQL. The application provides product listings, user login, shopping cart management, and image uploads.

After deployment to a Linux server, several features stop working.

This example demonstrates that a single website may experience several independent problems after deployment. Each problem requires investigation and an appropriate correction.

The developer should first identify the exact error, then check the most relevant configuration or component. After making a change, the affected feature should be tested again to confirm that the problem has been resolved.

Observed problem	Possible cause	Corrective action
Product page displays an error	Incorrect capitalization of an included PHP file	Correct the file reference
Product images are missing	Incorrect image path or file name	Verify the path and capitalization
Image uploads fail	Upload directory is not writable	Configure suitable ownership and permissions
Login fails	Incorrect database credentials	Update production database settings
Database functions are unavailable	Required PHP database extension is missing	Install and enable the extension
Large files cannot be uploaded	PHP upload limits are too low	Review upload configuration
Friendly URLs return 404	URL rewriting is missing or incorrect	Correct the Apache or Nginx configuration
Website displays a blank page	Fatal PHP error or incorrect configuration	Inspect PHP and web server logs
PDF reports cannot be generated	Missing library or extension	Install compatible dependencies
Website cannot send emails	Incorrect mail service configuration	Configure and test the mail service

5. Best Practices to Prevent Deployment Errors

The following practices help reduce compatibility problems:

- Develop and test the application in an environment similar to the production server.
- Use version control systems such as Git to track code changes.
- Maintain consistent PHP versions and dependencies.
- Follow consistent naming conventions for files and directories.
- Use secure file permissions.
- Store database credentials in environment variables rather than directly in source code.
- Test database connections and server configuration before deployment.
- Monitor application logs regularly.

6. Conclusion

A PHP website that works correctly on a developer's Windows computer but fails after deployment to a Linux server may be affected by several differences between the two environments. The most common causes include operating system differences, case-sensitive file names, incorrect paths, file permissions, incompatible PHP versions, missing extensions, configuration errors, database connectivity problems, web server settings, and missing dependencies.

The developer should identify the actual cause by examining error messages and logs, checking file names and permissions, comparing software versions, verifying configuration settings, and testing database connectivity. Each correction should be followed by appropriate testing to confirm that the application works as expected.

In addition, using version control, maintaining consistent dependencies, securing configuration files, testing in a production-like environment, and monitoring server logs can prevent many deployment problems.

Therefore, successful PHP deployment requires more than transferring application files from one computer to another. It requires ensuring that the production environment is correctly configured, compatible with the application, and secure. Following a systematic troubleshooting procedure helps developers achieve reliable website operation on Linux servers.

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimize code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrate s strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrate d
Code Quality	15%	Produces clean, well-structured code with proper comments and	Code is mostly clear with minor documentation gaps	Code lack's structure and proper documentation	Poorly written code with no documentation